



VIT[®]
BHOPAL

Name:Gauri Gupta

Reg.no.: 25BAI11091

Date: 18/Nov/25

Project Title: Shopping App

Introduction

The Shopping App is developed to provide users an easy and convenient platform to browse, select, and purchase Indian and Western wear. The app captures user details and offers a categorized shopping experience with a smooth flow from **product** selection to order confirmation.

Problem Statement

Many users find it difficult to browse and order ethnic and western clothing from multiple sources. This app aims to simplify the shopping process by integrating both categories and providing clear options along with a seamless user experience.

Functional Requirements

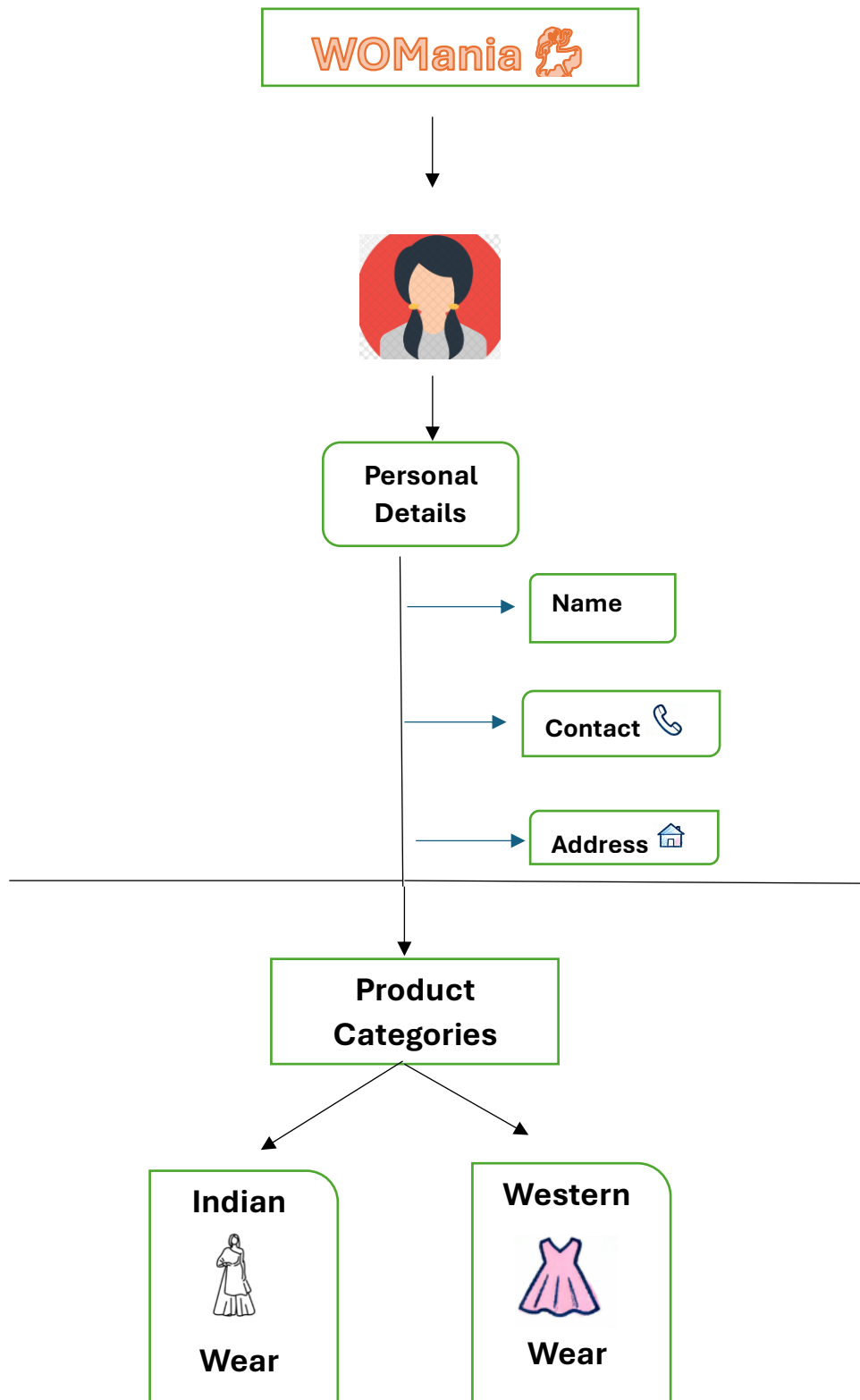
- User registration with name, mobile number, and contact number
- Choice selection between Indian and Western wear
- Detailed subcategories under each clothing type
- Product browsing and selection
- Shopping cart functionality
- Order confirmation and summary display

Non-functional Requirements

- User-friendly and intuitive interface
- Secure data handling and user authentication
- Fast response time in displaying products and processing orders
- Mobile and desktop responsiveness

System Architecture

The app follows a modular architecture with separate components for user management, product management, cart handling, and order processing. Data handling can be done via local storage or a database backend.



Sarees -1800rs

Kurtis-600rs

Lehenga-2000rs

Sherwani-600rs

Accessories-200rs



T-Shirts800rs

Jeans-800rs

Dresses-1000rs

Jackets-600rs

Accessories-200rs

Add to Cart



Bill Total

Place Order

Design Decisions & Rationale

Modular design was chosen for easier maintenance and scalability. Category separation enhances UI clarity. The flowchart ensures logical flow and usability. Security and performance were prioritized in choosing technology stack.

Implementation Details

The app was implemented primarily using Python .

```
class User:
```

```
    def __init__(self, user_id, name, contact, address):  
        self.user_id = user_id  
        self.name = name  
        self.contact = contact  
        self.address = address  
        self.cart = Cart(self)
```

```
class Category:
```

```
    def __init__(self, category_id, name):
```

```
self.category_id = category_id  
self.name = name
```

```
class Product:
```

```
def __init__(self, product_id, name, price, category):  
    self.product_id = product_id  
    self.name = name  
    self.price = price  
    self.category = category
```

```
class Cart:
```

```
def __init__(self, user):  
    self.user = user  
    self.items = [] # List of Product
```

```
def add_product(self, product):  
    self.items.append(product)
```

```
def view_cart(self):  
    for item in self.items:  
        print(f"{item.name} - ₹{item.price}")
```

```
def calculate_total(self):  
    return sum(item.price for item in self.items)
```

```
class Order:
```

```
    def __init__(self, order_id, user, cart):  
        self.order_id = order_id  
        self.user = user  
        self.items = cart.items  
        self.total = cart.calculate_total()
```

```
    def confirm_order(self):  
        print(f"Order Confirmed for {self.user.name}")  
        print(f"Total Bill: ₹{self.total}")
```

```
# app.py (Main Logic)
```

```
def main():
```

```
    # Sample categories and products
```

```
    indian_category = Category(1, "Indian Wear")
```

```
    western_category = Category(2, "Western Wear")
```

```
    products = [
```



```
Product(1, "Saree", 1200, indian_category),  
Product(2, "Kurti", 600, indian_category),  
Product(3, "Lehenga", 2500, indian_category),  
Product(4, "T-Shirt", 400, western_category),  
Product(5, "Jeans", 1000, western_category),  
]
```

```
# User Registration
```

```
user = User(101, "Amit Kumar", "9876543210", "Delhi")
```

```
# Choosing Category and Adding Products
```

```
print("Available Products:")
```

```
for idx, prod in enumerate(products, 1):
```

```
    print(f"{idx}. {prod.name} ({prod.category.name}) -  
₹{prod.price}")
```

```
# Simple add-to-cart (simulate user picks by index)
```

```
user.cart.add_product(products[0]) # Saree
```

```
user.cart.add_product(products[4]) # Jeans
```

```
# View cart and total
```

```
print("Your Cart:")
```

```
user.cart.view_cart()

print("Total Bill: ₹{user.cart.calculate_total()}")

# Place Order

order = Order(1001, user, user.cart)

order.confirm_order()

if __name__ == "__main__":
    main()
```

Screenshots

Available Products:

1. Saree (Indian Wear) - ₹1200
2. Kurti (Indian Wear) - ₹600
3. Lehenga (Indian Wear) - ₹2500
4. T-Shirt (Western Wear) - ₹400
5. Jeans (Western Wear) - ₹1000

Your Cart:

Saree - ₹1200

Jeans - ₹1000

Total Bill: ₹2200

Order Confirmed for Amit Kumar

Total Bill: ₹2200

Learnings & Key Takeaways

- Applying modular design improves project structure
- Flowcharts and diagrams greatly assist in understanding workflows
- Handling user data requires careful validation and error management

Future Enhancements

- Integration of payment gateway for real transactions
- User profile management with saved preferences and order history
- Adding advanced filters and search for products
- Implementing ratings and review sections
- Enabling push notifications for deals and order updates

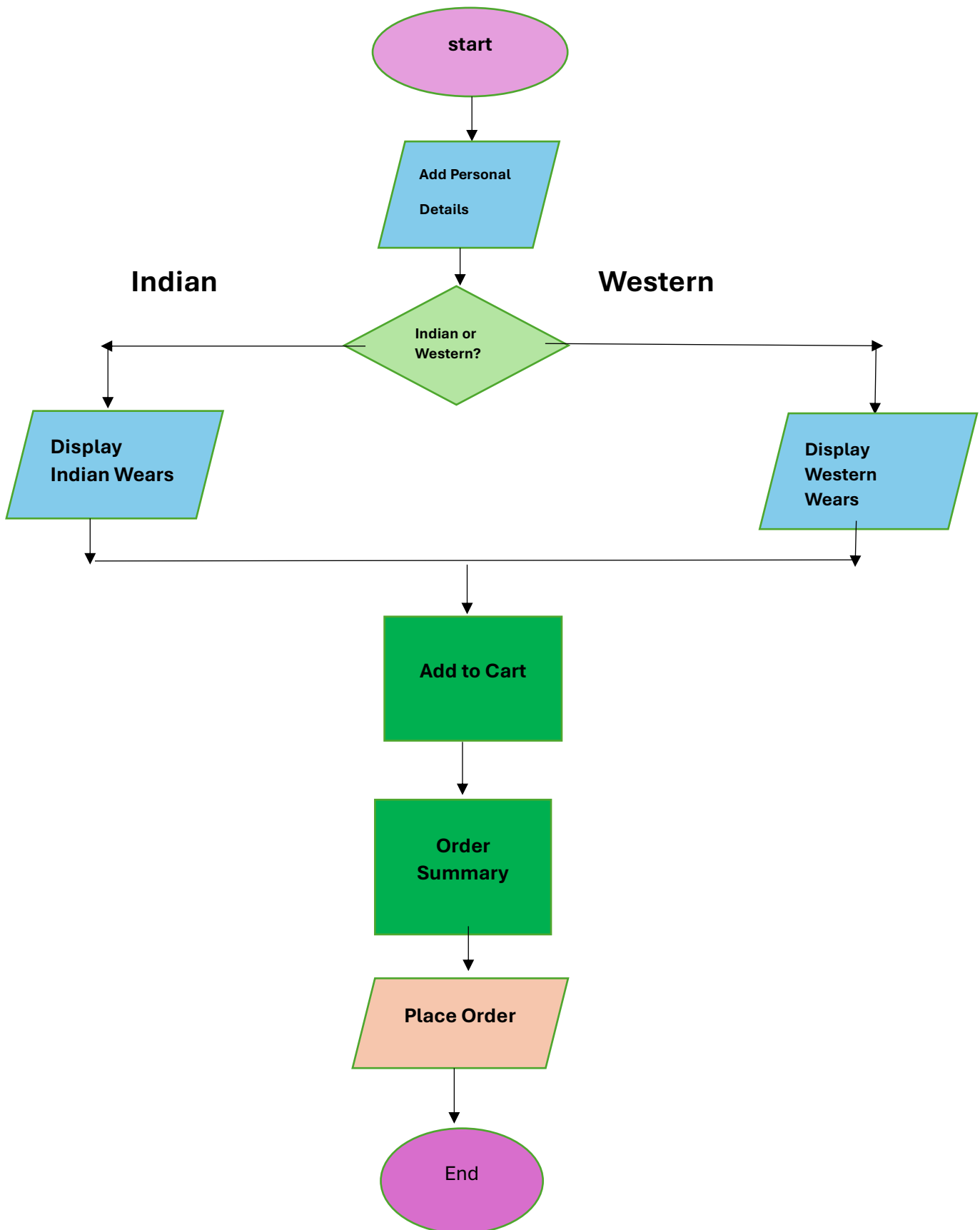
References

- Project guidelines document provided by course instructor
- Various online resources on Python programming

Design Diagrams

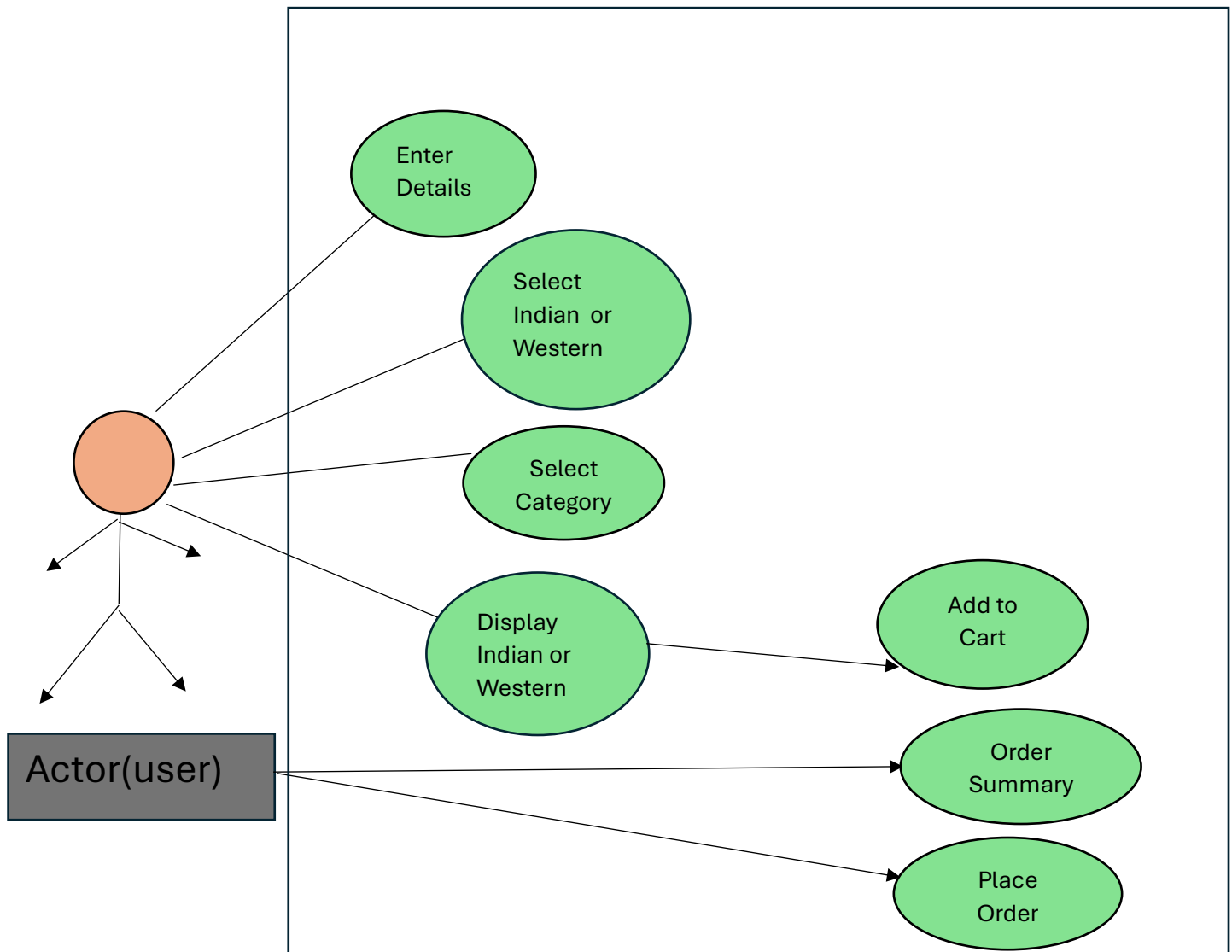
- **Use Case Diagram:** Users register/login, select categories, browse products, add to cart, and confirm orders.
- **Workflow Diagram:** See appended flowchart detailing stepwise process from user input to order confirmation.
- **Sequence Diagram:** Illustrates interaction between user, app interface, and backend for product selection and checkout.
- **Class/Component Diagram:** Classes including User, Product, Category, Cart, and Order modules.
- **ER Diagram:** (If database used) Entities like User, Product, Order with relationships.

Work Flow Diagram

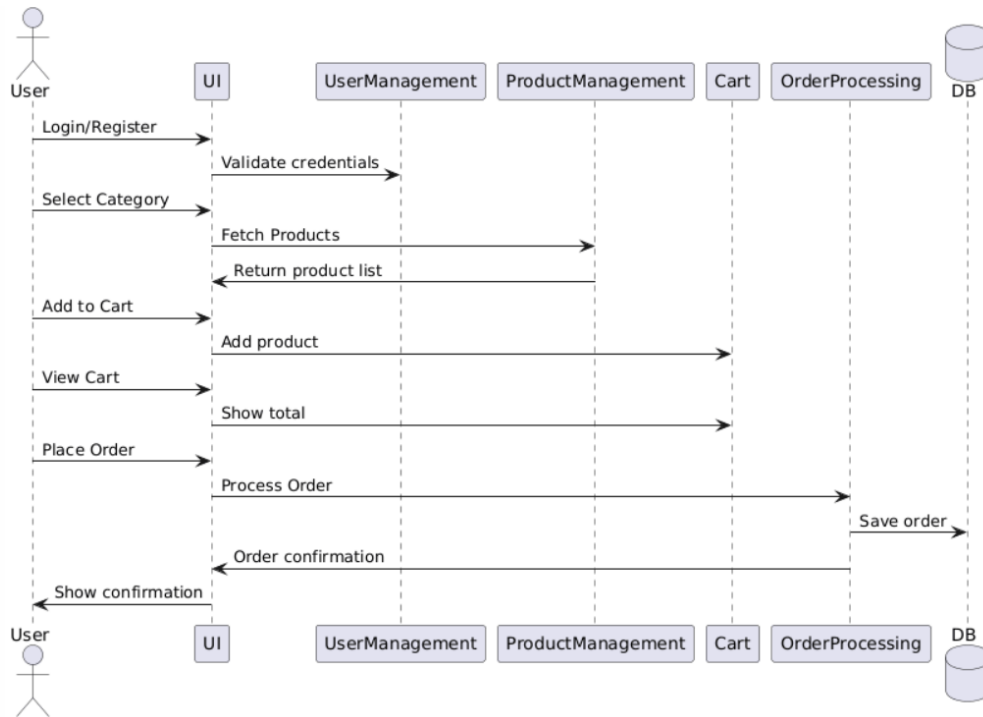


Use -Case Diagram

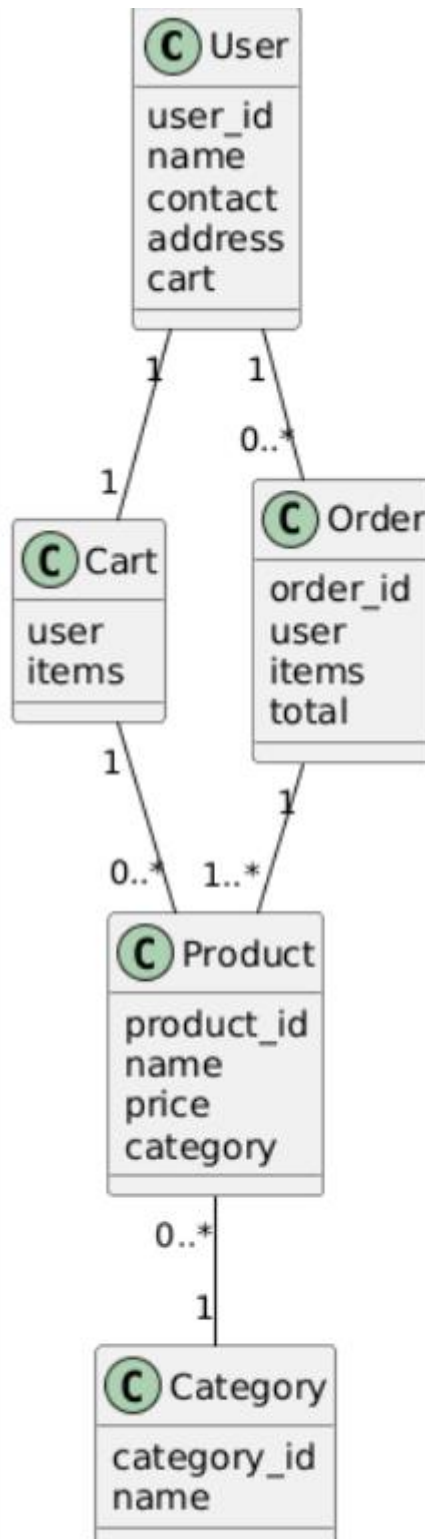
Shopping System



Sequence Diagram



ClassDiagram



Git hub repository

Below is given link of github repo code part ,readme.md and statement .md file

<https://github.com/gauri25bai11091-wq/Vityarthi.git>

References

- Project guidelines document provided by course instructor
- Various online resources on Python programming